

主机名、NSS 与 DNS 名称解析

从“这台主机叫什么”到“应用最终连向哪个地址”：把本机身份、来源顺序、搜索域、DNS 记录与应用证据放进同一条解析链。

对象模型操作语义验证诊断经典任务
大字号阅读版

本章阅读导航

先抓住一条主线：主机名是本机身份；名称解析是应用把名称转换为地址的过程。两者会相互影响，但任何一条命令都不能同时证明本机身份、DNS 记录、NSS 结果和应用功能。

01本机身份区分 static、transient 与 pretty hostname

02名称形式判断短名、FQDN 与绝对名称

03NSS 调度读取 hosts 数据库的来源和顺序

04本地与 DNS区分 /etc/hosts 与 A、AAAA、PTR

05客户端配置核对 DNS server、search 与有效 resolver

06最终证据用 getent、dig 与应用请求分层验收

专题地图

知识专题三种 hostname 与持久身份

知识专题短名、FQDN、绝对名称与 search

知识专题NSS hosts: 来源顺序

操作专题hostnamectl 与 /etc/hosts

知识专题A、AAAA 与 PTR

操作专题NetworkManager DNS 与 resolver

操作专题getent 与 dig 的证据边界

诊断专题短名失败、正反向不一致

诊断专题dig 成功但应用失败

经典任务配置任务与差异诊断任务

阅读时持续回答

1. 题目改变的是本机身份，还是名称到地址的映射？
2. 当前值与重启后持久值分别在哪里？
3. 应用提交的是短名、FQDN 还是绝对名称？
4. `hosts:` 行会先问哪个来源？
5. DNS server 和 search 来自哪个 profile？
6. 失败的是 A、AAAA 还是 PTR？
7. `dig` 与 `getent` 的差异排除了哪一层？
8. 哪条证据真正重复了原始失败路径？

使用方法：先用概念块区分对象，再通过操作语义确认接口，最后沿状态、查询、验证和诊断链完成任务。

经典任务与参考解答分别独立阅读。

第 19 章 · 正文

一台 Linux 主机“叫什么名字”，和一个应用“怎样把名字转换为地址”，是两个相互关联但并不相同的问题。主机名属于本机身份；`/etc/hosts` 是本地映射；NSS 决定系统库按什么来源和顺序查找；NetworkManager 保存 DNS server 与 search domain；DNS 服务端分别提供 A、AAAA、PTR 等记录。应用最终得到的结果还可能受自身缓存、长期连接或专用解析库影响。

最常见的误判都来自把这些对象压成一个“DNS 是否正常”的问题：`hostnamectl` 正确不代表 DNS 已发布；`dig` 成功不代表应用通过 NSS 得到同一地址；A 记录存在不代表 PTR 已建立；`/etc/resolv.conf` 当前正确不代表持久 profile 已正确；修改 `/etc/hosts` 也不会把结果传播给其他主机。

本章按照“身份 → 名称形式 → NSS 来源 → 本地或 DNS 数据 → 客户端配置 → 系统查询 → 应用功能”推进。IP 地址、连接 profile 的一般创建与路由属于第 18 章；SSH 主机密钥与身份校验属于第 20 章；本章不部署权威或递归 DNS 服务器。

贯穿本章的主问题

- 系统现在叫什么，重启后又会叫什么？
- 应用查询的名称会被怎样补全、按什么来源查找？
- DNS 服务端返回什么，系统 NSS 最终又返回什么？
- 哪一层证据成功，仍然不能证明下一层成功？

核心概念

概念

主机名的三种状态 `static hostname` 表达持久机器身份，通常落在 `/etc/hostname`；`transient hostname` 表达当前运行期可能由网络提供的临时名称；`pretty hostname` 是面向人的描述文本。三者解决的问题不同，不能因为 `hostnamectl status` 中某一项正确，就推断另外两项、DNS 记录或应用解析已经正确。

概念

本机身份与 DNS 名称 主机名描述本机如何标识自己，DNS 名称描述分布式名称空间中的节点。二者常被配置成相同字符串，但没有自动同步关系：`hostnamectl set-hostname` 不会创建 A、AAAA 或 PTR，DNS 服务端的记录也不会自动改写本机 `/etc/hostname`。

概念

NSS (Name Service Switch) 是应用通过系统库查询主机、用户、组等数据库时使用的来源调度机制。对主机名称而言，`/etc/nsswitch.conf` 的 `hosts:` 行决定可能访问 `files`、`dns`、`myhostname`、`resolve` 等来源及其顺序。NSS 是“应用通常怎么查”的主线，不能用直接 DNS 工具代替。

概念

DNS resolver 是客户端把 DNS 查询送往哪台服务器、怎样重试和怎样处理搜索域的运行入口。NetworkManager connection profile 通常是持久配置源，`/etc/resolv.conf` 是当前有效视图或指向本地 stub 的入口。只检查其中一层，无法证明持久配置、当前上游和最终查询同时正确。

概念

短名、FQDN 与 search domain `node1` 是需要上下文的短名；`node1.training.example` 已包含所属域；末尾带点的 `node1.training.example.` 明确表示绝对 DNS 名。search domain 只负责把短名扩展成待查询名称，不会创建记录，也不能保证不同客户端对同一短名得到相同目标。

概念

A、AAAA 与 PTR 是独立的 DNS 记录方向：A 把名称映射到 IPv4，AAAA 把名称映射到 IPv6，PTR 在反向命名空间中把地址映射到名称。正向成功并不推导反向成功；A 存在也不推导 AAAA 存在。诊断时必须明确

查询类型，而不是只说“DNS 有结果”。

概念

系统解析结果与应用结果 `getent` 观察系统 NSS 路径，`dig` 观察 DNS 查询，应用还可能使用缓存、专用运行时或已经建立的连接。因此最末层验收必须重复应用原始失败路径；工具成功是重要证据，但不能自动扩大成业务终态正确。

操作语义速查

操作语义先建立六个关键入口的作用对象和证据边界，再进入正文中的具体操作、验证和诊断。

hostnamectl

SYNOPSIS

```
hostnamectl [OPTIONS...] COMMAND [NAME]
```

观察或修改 systemd-hostnamed 管理的主机名状态。它回答本机身份，不直接查询 DNS。

hostnamectl status

同时查看 static、transient、pretty 等状态。

hostnamectl set-hostname NAME

设置题目要求的主机名；默认行为仍要结合当前版本帮助确认。

--static / --transient / --pretty

有意只观察或修改某一种状态时使用。

/etc/hosts

LINE FORM

```
IP-address primary-name [alias ...]
```

提供本机 `files` 来源的静态名称映射，适合小范围、节点私有或故障隔离，不会向网络发布 DNS。

192.0.2.44 repo.training.example repo-local

地址在前，主要名称随后，零个或多个别名位于末尾。

最小修改

修改前先查同名、同地址与 IPv4/IPv6 冲突；不要删除无关条目。

验证

使用 `getent` 证明 NSS 结果，不使用 `dig` 证明本地条目。

/etc/nsswitch.conf • hosts:

POLICY FORM

```
hosts: files dns
hosts: files myhostname dns
```

规定系统主机数据库的来源顺序和可选状态动作。不要背诵一条“永久默认值”，应读取当前系统真实行。

files

通常读取 `/etc/hosts`。

dns

通过当前 resolver 查询 DNS。

[STATUS=ACTION]

高级状态动作会改变继续或停止策略；修改前必须先读手册和现状。

getent

SYNOPSIS

```
getent database [key ...]
```

通过系统 NSS 查询数据库，是观察普通应用系统解析路径的主要入口。

```
getent hosts NAME
```

查询主机数据库的常用形式。

```
getent ahostsv4 NAME
```

只观察 IPv4 地址族结果。

```
getent ahostsv6 NAME
```

只观察 IPv6 地址族结果。

dig

SYNOPSIS

```
dig [@server] name [type] [query-option ...]
```

直接构造 DNS 查询并显示服务端回答。它能隔离 DNS 层，但不会读取 `/etc/hosts`，也不能代表完整 NSS 或应用行为。

```
dig NAME A / AAAA
```

分别查询 IPv4 与 IPv6 正向记录。

```
dig -x ADDRESS
```

构造反向 PTR 查询。

```
dig @SERVER NAME TYPE
```

隔离指定 DNS 服务器。

```
+short
```

只显示简短答案；排错时仍应保留完整状态与 ANSWER 证据。

+search

显式使用 resolver 搜索列表测试短名。

nmcli • DNS 属性

SYNOPSIS

```
nmcli connection show [NAME]
nmcli connection modify NAME PROPERTY VALUE
```

修改 NetworkManager connection profile 中的持久 DNS 输入。IP 地址和路由的一般配置留在第 18 章。

ipv4.dns / ipv6.dns

保存对应地址族的 DNS server 列表。

ipv4.dns-search / ipv6.dns-search

保存搜索域。

ipv4.ignore-auto-dns yes

只有题意明确要求排除 DHCP 等自动 DNS 时才使用。

nmcli connection up NAME

重新激活 profile 可能中断远程会话；操作前先评估连接路径。

名称解析链：每一层回答不同问题

本机 hostname→名称形式→NSS hosts 策略→/etc/hosts 或 DNS→A / AAAA / PTR→应用结果

层次	首要证据	能证明什么	不能单独证明什么
本机身份	hostnamectl、 hostname、/etc/hostname	当前或持久主机名	DNS 已有同名记录
NSS 策略	grep '^hosts:' /etc/nsswitch.conf	来源与顺序	每个应用都没有自有缓存
本地映射	/etc/hosts、getent	本机 files 结果	其他主机也能解析
DNS 客户端	nmcli、/etc/resolv.conf	持久输入与当前入口	指定记录一定存在
DNS 数据	dig	某服务器对某类型查询的回答	NSS 或应用得到同一结果
应用功能	原始应用请求与日志	最终路径是否恢复	所有其他客户端同时恢复

[知识专题] 一台主机为什么同时存在三种 hostname

主机名不是单一字符串。systemd 将它拆成静态、临时和美化三种视图，是为了分别表达持久机器身份、当前运行名称和人类可读描述。实际调查时，最顺的入口是先运行 `hostnamectl status`，再确认 `/etc/hostname`，最后判断当前名称是否经过 NSS 能形成期望的 FQDN。

① [知识点] static hostname 是重启后应恢复的持久身份

静态主机名通常保存在 `/etc/hostname`。在 RHEL 9 中，应优先通过 `hostnamectl set-hostname` 修改，而不是把“编辑文件成功”当作完整操作。静态主机名适合使用可由 DNS 或本地映射解析的名称，例如：

```
node1.training.example
```

静态主机名回答的是：这台机器被持久配置成什么名称？它不直接回答当前 DNS 中是否已有同名 A、AAAA 或 PTR 记录。

② [知识点] transient hostname 表示当前运行期的临时名称

临时主机名存在于当前运行状态中，可能由内核启动参数、DHCP 或其他网络配置提供。当静态主机名未正确设置时，临时名称可能成为当前显示值。它通常不应被当作重启后的唯一保证。

调查时要避免只执行一次 `hostname` 就下结论。`hostname` 主要反映当前内核名称，而 `/etc/hostname` 反映持久静态配置。二者不一致时，应先识别题目要求的是“现在改变”还是“重启后保持”。

③ [知识点] pretty hostname 是描述，不是 DNS 标签

美化主机名用于管理界面或资产描述，可以包含空格和大小写，例如：

```
Training Web Node 1
```

它不应被放入 `/etc/hosts` 或 DNS zone，也不能作为服务端点的稳定标识。考试和日常运维涉及可解析主机身份时，重点通常是静态主机名。

④ [知识点] 本机主机名与 FQDN 不是自动等价关系

FQDN 是完整限定名称。系统当前主机名可能本身就是 FQDN，也可能只是短名。类似 `hostname -f` 的接口会尝试结合当前主机名和名称解析结果得到规范名称，因此其结果依赖 NSS、`/etc/hosts` 和 DNS，而不是单纯读取 `/etc/hostname`。

因此：

```
hostnamectl 显示 node1.training.example
```

只能证明本机身份已经设置；仍需用 `getent hosts node1.training.example` 或应用实际访问确认解析链。

⑤ [知识点] 修改 hostname 不会发布 DNS

`hostnamectl set-hostname` 改变本机状态，但不会：

- 向权威 DNS 创建 A 或 AAAA 记录；
- 创建 PTR 记录；
- 修改其他客户端的 `/etc/hosts`；
- 自动更新 SSH `known_hosts`；
- 证明依赖旧主机名的应用配置已经迁移。

在真实工作中，主机名变更应与 DNS、监控、证书、日志标签和资产平台一起评估。本章只完整处理本机身份和客户端解析链。

Cheatsheet static 看持久身份；transient 看当前临时值；pretty 只作描述；`hostname` 看当前内核名称；``/etc/hostname`` 看持久真源；设置主机名不等于发布 DNS。

[知识专题] 短名、FQDN、绝对名称与搜索域

同一个目标可能以 `node1`、`node1.training.example` 或 `node1.training.example.` 出现。这些字符串不是简单的长短差异：它们决定解析器是否使用搜索域、怎样生成查询序列，以及诊断时是否存在歧义。最稳妥的切入路径是分别测试短名、FQDN 和带末尾点的绝对名称。

① [知识点] 短名需要上下文才能成为完整名称

`node1` 没有指出所属 DNS 域。系统解析器可能把它与 search domain 组合：

```
node1 + training.example  
→ node1.training.example
```

短名适合在边界明确的内部环境中使用，但它的含义依赖客户端配置。不同客户端拥有不同 search 列表时，同一个短名可能解析到不同目标。

② [知识点] FQDN 表达从主机标签到 DNS 域的完整路径

`node1.training.example` 通常被人理解为 FQDN。严格地说，DNS 绝对名称以根标签结束，文本形式可以写为：

```
node1.training.example.
```

末尾点能够明确表示“不再追加搜索域”。日常命令通常省略它；诊断多搜索域或可疑多标签名称时，带末尾点的查询更有区分度。

③ [知识点] search domain 只解决名称补全，不决定 DNS 服务器内容

`search training.example` 告诉解析器怎样扩展短名，但不会创建任何记录。要让 `node1` 成功，扩展后的名称仍必须在 `/etc/hosts`、DNS 或其他 NSS 来源中存在。

`search` 列表按顺序尝试各搜索域。列表越长，短名失败可能产生越多查询，也更容易因同名资源导致歧义。

`/etc/resolv.conf` 还支持 `domain example.com` 表示单一搜索域；`search` 则可列出按顺序尝试的多个域。如果文件中同时出现 `domain` 与 `search`，应以最后出现的有效指令所形成的搜索列表为准。实际运维中仍要回到 NetworkManager profile 确认持久来源，而不是只改当前文件。

④ [知识点] ndots 会影响多标签名称的尝试顺序

`resolv.conf` 的 `options ndots:N` 影响包含多少个点的名称会优先按绝对名称尝试。默认行为和具体失败回退由 resolver 实现决定。RHCSA 操作不要求背诵所有算法分支，但必须知道：

- 无点短名通常依赖 `search`；
- 多标签名称不一定完全绕过 `search`；
- 带末尾点的绝对名称可消除搜索扩展歧义；
- 诊断时不要只测试一种写法。

⑤ [知识点] dig 默认不等同于应用的短名解析

`getent hosts node1` 走 NSS 和系统 resolver；`dig node1` 面向 DNS 层，默认行为不应被当作应用查询的完整等价物。需要显式测试 `search` 时，可使用：

```
dig +search node1 A
```

更可控的做法是先从 `/etc/resolv.conf` 或 NetworkManager 读取 `search domain`，再直接查询构造出的 FQDN。

Cheatsheet 短名依赖 `search`；FQDN 减少歧义；末尾点表示绝对名称；`search` 不创建记录；应用短名验证优先用 ``getent``，DNS 记录验证优先用 ``dig``。

[知识专题] NSS 的 hosts 数据库如何选择解析来源

应用通常不会自己逐个读取 `/etc/hosts`、再访问 DNS。它调用系统解析接口，由 NSS 根据 `/etc/nsswitch.conf` 中的 `hosts:` 行选择来源。排错的关键不是背诵某个默认行，而是读取当前系统的真实配置并理解“顺序”和“返回状态”。

① [知识点] hosts: 是数据库策略，不是 DNS 配置

一个常见但不能当作所有系统绝对默认值的示例是：

```
hosts: files dns
```

含义是先通过 `files` 来源查找主机数据库，再进入 `dns`。其中：

- `files` 通常对应 `/etc/hosts`；
- `dns` 通过 `resolver` 配置查询 DNS；
- `myhostname` 可合成本机名称和本地地址；
- `resolve` 可把请求交给 `systemd-resolved`；
- 其他模块是否存在取决于已安装软件和当前配置。

② [知识点] 来源顺序决定冲突时谁先提供答案

如果 `/etc/hosts` 中有旧地址，而 `files` 位于 `dns` 之前，`getent hosts` 和多数系统应用可能优先取得旧地址，即使 `dig` 显示 DNS 已经正确。

这不是“DNS 缓存一定没清”，而是两个工具查询了不同路径。正确修复通常是处理过期本地条目，而不是把 `dns` 移到 `files` 前面掩盖冲突。

③ [知识点] NSS 还可以根据返回状态决定继续或停止

`nsswitch.conf` 支持类似以下动作语法：

```
[STATUS=ACTION]
```

状态可能包括 `SUCCESS`、`NOTFOUND`、`UNAVAIL`、`TRYAGAIN`，动作通常是 `return` 或 `continue`。本章不要穷尽所有模块组合，但要能识别：中括号动作可能让解析链在某种失败状态下提前停止。

修改这类全局策略风险较高。没有明确任务和证据时，不应为了让一次查询成功而改变整个 `hosts:` 行的全局来源顺序。

④ [知识点] `getent` 是观察 NSS 结果的主要接口

```
getent hosts node1.training.example
```

它调用系统数据库接口，因此比 `cat /etc/hosts` 更接近应用最终看到的结果。需要区分地址族时，可使用：

```
getent ahostsv4 node1.training.example
getent ahostsv6 node1.training.example
```

输出可能包含多行、多地址或不同排序。判断重点是是否得到预期地址族和地址，而不是背诵固定行序。

⑤ [知识点] NSS 成功仍不代表应用业务成功

`getent` 成功证明系统解析接口当前能得到结果，但应用仍可能：

- 使用自己的 DNS 客户端库；

- 缓存旧地址；
- 已经保持旧连接；
- 在容器或独立网络命名空间中使用不同 resolver；
- 因端口、路由、TLS 或服务端状态失败。

因此名称解析通过后，仍需执行应用功能验证。

Cheatsheet 先读真实 ``hosts:`` 行；顺序决定冲突优先级；中括号动作可能提前停止；``getent`` 观察 NSS；NSS 成功不等于业务终态。

[操作专题] 查询和设置主机名并区分当前与持久状态

主机名变更看似简单，但至少包含当前状态、静态持久配置和名称解析三个维度。安全操作从建立基线开始，随后只修改题目要求的类型，再分别验证当前和持久状态。

① [查询] 建立主机名基线

作用对象：systemd-hostnamed 管理的 hostname 状态和 `/etc/hostname`。

基本形式：

```
hostnamectl status
hostname
cat /etc/hostname
```

较新 `hostnamectl` 版本可用类型选项配合 `hostname` 子命令隔离查询；RHEL 9 考试中以 `hostnamectl status` 和持久文件交叉验证即可。不要把 `pretty hostname` 当作可解析名称。

② [操作] 设置题目要求的静态主机名

典型形式：

```
hostnamectl set-hostname node1.training.example
```

该操作通常立即更新当前名称并写入静态配置。若只想修改一种类型，应显式使用对应选项，例如：

```
hostnamectl --pretty set-hostname "Training Web Node 1"
```

边界：具体版本对默认同时设置哪些 `hostname` 类型的表现，应以 `hostnamectl status` 验证，不靠记忆代替证据。

③ [操作] 验证当前状态与持久状态

```
hostname
hostnamectl status
cat /etc/hostname
```

验证解释：

- `hostname`：当前内核名称；
- `hostnamectl status`：systemd 管理的各类名称与系统信息；
- `/etc/hostname`：静态持久配置。

若题目明确要求重启后保持，静态配置检查只能作为持久性证据；最终仍需在目标系统上真实重启，再重复检查当前状态、名称解析和应用功能。

④ [操作] 验证主机名是否能沿 NSS 解析

```
getent hosts node1.training.example
```

如果当前主机名只设置了身份但没有本地或 DNS 记录，`getent` 可能失败。此时应进入 `/etc/hosts`、NSS 和 DNS 调查，而不是重复执行 `hostnamectl`。

⑤ [操作] 变更前后记录最小证据

建议保存：

```
hostnamectl status
cat /etc/hostname
grep -E '^[[:space:]]*hosts:' /etc/nsswitch.conf
getent hosts "${hostname}"
```

这些命令分别记录身份、持久真源、解析策略和当前查询结果。生产环境还应检查证书、监控标签和集群配置，但这些扩展不作为本章考试主线。

Cheatsheet `hostnamectl set-hostname` 修改；`hostname` 看当前；`/etc/hostname` 看持久；`getent` 看能否解析；身份正确不代表 DNS 正确。

[操作专题] 用 `/etc/hosts` 建立可审计的本地名称映射

`/etc/hosts` 适合少量、明确、本机范围的静态映射。它不是“小型 DNS 服务器”，也不会自动传播。高质量操作不是简单追加一行，而是先查冲突、使用规范字段顺序、再通过 NSS 验证。

① [知识点] 每行由地址、规范名和可选别名组成

```
192.0.2.44 repo.training.example repo-local
```

字段含义：

1. 地址；
2. 规范名称；
3. 零个或多个别名。

空白可由空格或制表符分隔，`#` 后为注释。IPv4 和 IPv6 映射应分别写明，不要把一个地址族的记录当作另一个地址族的替代。

② [查询] 修改前先寻找同名和同地址条目

```
grep -nE '(^|[:space:]]repo-local([[:space:]]|$)' /etc/hosts
grep -nF 'repo.training.example' /etc/hosts
grep -nF '192.0.2.44' /etc/hosts
```

调查目标包括：

- 同名是否指向多个不同地址；
- 旧规范名是否仍保留；
- 别名是否重复；
- 是否存在 IPv4 与 IPv6 的不同预期；
- 是否误改 localhost 基础条目。

③ [操作] 使用安全编辑方式完成最小修改

考试中可使用熟悉的文本编辑器。真实工作中应先备份或使用配置管理。不要用无条件 `echo >> /etc/hosts` 反复追加，因为重入后容易产生重复和冲突。

示例目标行：

```
192.0.2.44 repo.training.example repo-local
```

④ [操作] 通过 `getent` 而不是 `dig` 验证本地映射

```
getent hosts repo.training.example
getent hosts repo-local
```

`dig` 和 `host` 面向 DNS，不会证明 `/etc/hosts` 条目被 NSS 使用。`cat /etc/hosts` 只能证明文件内容存在，也不能证明 `hosts:` 顺序允许该来源返回答案。

⑤ [边界] 本地映射不会发布到其他节点

`/etc/hosts` 只影响读取该文件并遵循相应 NSS 策略的本机环境。它不提供：

- 集中更新；
- TTL 与动态记录；
- 远端客户端查询；
- 自动 PTR；
- 容器内独立 `hosts` 文件的同步。

当映射数量、节点数量或变更频率上升时，应使用集中 DNS 或配置管理，而不是无限扩展手工 `hosts` 文件。

Cheatsheet 地址在前、规范名其次、别名最后；修改前查冲突；验证用 ``getent``；``dig`` 不读 ``/etc/hosts``；本地映射不会传播。

[知识专题] A、AAAA 与 PTR 是三类独立 DNS 数据

DNS 不是简单的“名称与 IP 对照表”。查询必须说明名称、记录类型和服务器。A、AAAA 与 PTR 分别处理 IPv4 正向、IPv6 正向和地址到名称的反向映射，它们可以独立存在、独立出错。

① [知识点] A 记录把名称映射到 IPv4 地址

```
dig node1.training.example A
```

一项名称可以返回多个 A 记录。多地址可能用于负载分担或容错，并不天然表示冲突。验证时应判断题目是否要求“包含某地址”还是“只能有某地址”。

② [知识点] AAAA 记录把名称映射到 IPv6 地址

```
dig node1.training.example AAAA
```

A 成功而 AAAA 没有答案，不等于 DNS 整体失败。是否需要 AAAA 取决于题目、网络栈和应用。排错时应显式指定记录类型，避免默认查询掩盖地址族差异。

③ [知识点] PTR 记录把地址映射到名称

```
dig -x 192.0.2.44
```

`dig -x` 会构造反向区域名称并查询 PTR。IPv4 使用 `in-addr.arpa`，IPv6 使用 `ip6.arpa`。PTR 所属区域可能由不同团队或网络提供方维护，因此正向记录正确不能保证反向记录存在。

④ [知识点] 正向和反向不要求机械一一对应

一个名称可以有多个地址；多个名称也可能指向同一地址。某些服务依赖反向解析或正反向一致性，但这属于应用策略，不能把它当作 DNS 协议对所有记录的自动约束。

验证矩阵应分别记录：

```
名称 --A----> IPv4
名称 --AAAA-> IPv6
地址 --PTR--> 名称
```

⑤ [知识点] DNS 状态和 ANSWER 数量必须一起解释

常见响应判断：

- NOERROR 且有 ANSWER：该类型有数据；
- NOERROR 但 ANSWER 为 0：名称可能存在，但该类型没有数据；
- NXDOMAIN：被查询名称不存在；
- SERVFAIL：服务器无法完成解析；
- REFUSED：服务器拒绝该查询；
- 超时：还需区分网络不可达、端口过滤和服务器无响应。

只看到 NOERROR 就写“解析正常”是不完整的。

Cheatsheet A 查 IPv4；AAAA 查 IPv6；`dig -x` 查 PTR；正反向独立；状态码要和 ANSWER 一起看。

[操作专题] 通过 NetworkManager 配置 DNS server 与 search domain

在 RHEL 9 默认网络栈中，持久 DNS 配置通常属于 NetworkManager connection profile。`/etc/resolv.conf` 是当前有效视图，可能由 NetworkManager 生成，也可能指向本地 stub。正确路径是先识别 profile 和文件所有权，再修改 profile，最后重新激活并分层验证。

① [查询] 识别活动连接、设备和 resolver 形态

```
nmcli connection show --active
nmcli device status
readlink -f /etc/resolv.conf
cat /etc/resolv.conf
```

不要只看到 nameserver 127.0.0.1 或 127.0.0.53 就判断上游 DNS 丢失。该地址可能是本地缓存或 stub resolver，需要继续检查当前系统实际使用的组件。

② [查询] 读取 profile 中的持久 DNS 属性

```
nmcli -f connection.id,connection.interface-name,ipv4.method,ipv4.dns,ipv4.dns-search,ipv4.ignore-auto-dns \
connection show exam-net
```

IPv6 环境还应读取：

```
nmcli -f ipv6.method,ipv6.dns,ipv6.dns-search,ipv6.ignore-auto-dns \
connection show exam-net
```

这里查看的是保存配置，不等同于当前设备已经应用。

③ [操作] 替换、追加或删除 DNS server

替换当前 IPv4 DNS 列表：

```
nmcli connection modify exam-net ipv4.dns "192.0.2.53"
```

追加一个服务器：

```
nmcli connection modify exam-net +ipv4.dns "192.0.2.54"
```

删除一个服务器：

```
nmcli connection modify exam-net -ipv4.dns "192.0.2.54"
```

在修改前必须确认题意是替换还是追加。误用默认替换会删除原有备用服务器；无条件追加又可能保留题目明确要求移除的地址。

④ [操作] 配置 search domain

```
nmcli connection modify exam-net ipv4.dns-search "training.example"
```

多个搜索域是多值属性，应在修改后读取 profile 确认顺序。search 只补全短名，不创建记录，也不会修复错误的 `/etc/hosts`。

⑤ [操作] 仅在题意明确时忽略自动 DNS

DHCP 或其他自动配置可能提供 DNS。题目若要求“只使用指定服务器”，可设置：

```
nmcli connection modify exam-net ipv4.ignore-auto-dns yes
```

如果题目只是要求增加服务器，不能机械地启用该选项，因为它会丢弃自动获得的 DNS 和搜索域。IPv6 自动 DNS 需要独立判断 `ipv6.ignore-auto-dns`。

⑥ [操作] 应用配置并控制中断风险

常见考试操作：

```
nmcli connection up exam-net
```

重新激活可能短暂影响远程连接。真实工作中应通过控制台、维护窗口或评估 `nmcli device reapply` 的适用性。不要在未确认 connection 与接口关系时随意 down 全部连接。

⑦ [验证] 分开检查持久 profile 与当前设备状态

```
nmcli -f ipv4.dns,ipv4.dns-search,ipv4.ignore-auto-dns connection show exam-net
nmcli -f GENERAL.CONNECTION,IP4.DNS,IP4.DOMAIN device show
cat /etc/resolv.conf
```

验证解释：

- connection view：重启后应恢复的配置；
- device view：当前活动设备得到的 DNS 信息；
- resolver 文件或 stub：系统解析器当前入口；
- `getent / dig`：实际查询行为。

⑧ [边界] 不把直接锁定 `/etc/resolv.conf` 当作标准答案

直接编辑可能被 NetworkManager 在连接重激活时覆盖；使用 `chattr +i` 则可能破坏网络管理器的正常更新。除非系统明确配置为手工管理模式，否则应回到 connection profile 修改。

Cheatsheet profile 是持久真源；device view 是当前状态；`/etc/resolv.conf` 是有效入口；`ipv4.dns` 替换，`+` 追加，`-` 删除；`ignore-auto-dns` 只在明确要求时使用。

[操作专题] 用 `getent` 与 `dig` 分别验证系统解析和 DNS

名称解析排错最有价值的对照，是对同一个名称分别运行 `getent` 与 `dig`。两者结果相同可以缩小问题范围；两者结果不同则直接暴露 NSS、本地映射、resolver 配置或指定服务器之间的分层差异。

① [查询] 用 `getent hosts` 模拟系统应用的 NSS 路径

```
getent hosts node1.training.example
```

短名验证：

```
getent hosts node1
```

地址族验证：

```
getent ahostsv4 node1.training.example  
getent ahostsv6 node1.training.example
```

`getent` 返回成功只证明当前系统数据库接口得到了结果。输出可能包含多行，地址排序也可能随策略变化。

② [查询] 用 `dig` 显式指定记录类型

```
dig node1.training.example A  
dig node1.training.example AAAA  
dig -x 192.0.2.44
```

简洁查看答案可使用：

```
dig +short node1.training.example A
```

但 `+short` 隐藏状态、权威信息和失败原因。诊断失败时应回到完整输出。

③ [查询] 用 `@SERVER` 隔离特定 DNS 服务器

```
dig @192.0.2.53 node1.training.example A
```

这条命令回答：指定服务器怎样响应？它会绕开系统默认 `nameserver` 选择，不能证明当前应用已经使用该服务器。

若指定服务器成功而普通 `dig` 失败，下一步应检查 `/etc/resolv.conf`、本地 `stub`、`NetworkManager` 当前 DNS 顺序和网络可达性。

④ [查询] 显式测试 `search domain`

系统短名路径：

```
getent hosts node1
```

DNS 工具显式启用搜索：

```
dig +search node1 A
```

无歧义绝对查询：

```
dig node1.training.example. A
```

三种命令回答不同问题，应在报告中保留实际查询字符串。

⑤ [验证] 建立最小名称解析矩阵

目标	推荐证据	主要证明
本地/NSS FQDN	<code>getent hosts FQDN</code>	系统解析路径
NSS 短名	<code>getent hosts SHORT</code>	search 与 NSS 合成结果
IPv4 DNS	<code>dig FQDN A</code>	默认 DNS 路径的 A 记录
IPv6 DNS	<code>dig FQDN AAAA</code>	默认 DNS 路径的 AAAA 记录
指定服务器	<code>dig @SERVER FQDN A</code>	单台 DNS server 的回答
反向记录	<code>dig -x ADDRESS</code>	PTR 结果
应用终态	应用请求与日志	实际功能

⑥ [边界] 不使用 ping 代替名称解析验收

`ping name` 可能调用系统解析器，但后续还依赖 ICMP、路由、防火墙和对端响应。ping 失败不能直接证明 DNS 失败；ping 成功也不能证明目标服务端口可用。

Cheatsheet ``getent`` 看 NSS；``dig`` 看 DNS；``@SERVER`` 只证明指定服务器；``+short`` 适合取值、不适合解释失败；最终仍需应用验证。

[诊断专题] 短名失败、FQDN 成功与正反向不一致

这类故障的高区分度证据不是“再换一个 DNS”，而是把名称形式和记录类型拆开。诊断按症状、证据、假设、最小修复和再验证推进。

① [诊断] 短名失败而 FQDN 成功

症状：

```
getent hosts node1          失败
getent hosts node1.training.example 成功
```

假设：search domain 缺失、顺序错误，或应用没有使用系统 resolver。

下一条证据：

```
cat /etc/resolv.conf
nmcli -f ipv4.dns-search,ipv6.dns-search connection show exam-net
nmcli -f IP4.DOMAIN,IP6.DOMAIN device show
```

最小修复：在正确的现有 profile 上配置所需 search domain，重新应用。

再验证：同时重复短名和 FQDN 的 `getent`，不能只看配置文件。

② [诊断] `hostnamectl` 正确而 `hostname -f` 异常

当前证据：静态主机名已经设置，但规范名称派生失败或返回意外值。

假设：当前 `hostname` 无法在 NSS 中映射，或 `/etc/hosts` 的规范名和别名顺序不符合预期。

下一条证据：

```
hostname
getent hosts "${hostname}"
grep -nF "${hostname}" /etc/hosts
grep -E '^[[:space:]]*hosts:' /etc/nsswitch.conf
```

最小修复：处理本地映射或 DNS 记录，不重复设置已经正确的 static `hostname`。

③ [诊断] A 成功而 AAAA 没有数据

当前证据：

```
dig node1.training.example A
dig node1.training.example AAAA
```

如果 A 有答案而 AAAA 返回 `NOERROR`、ANSWER 为 0，应先确认题目和应用是否要求 IPv6。没有 AAAA 可能是预期状态，不应直接修改 IPv4 配置或宣布 DNS 整体故障。

④ [诊断] 正向成功而反向 PTR 失败

当前证据：A 或 AAAA 正确，`dig -x` 返回 `NXDOMAIN`、空答案或其他错误。

假设：反向区域未委派、PTR 未创建或由另一管理方维护。

最小修复边界：客户端只能证明问题并提供地址、期望名称和服务器证据；部署或修改权威 DNS 不在本章范围。不要通过本机 `/etc/hosts` 伪造“网络反向解析已经修复”。

⑤ [诊断] 多个 search domain 导致短名命中错误目标

症状：短名能解析，但得到的 FQDN 或地址不是预期目标。

下一条证据：

```
cat /etc/resolv.conf
getent hosts node1
dig node1.first.example. A
dig node1.second.example. A
```

最小修复：调整正确 profile 的 search 顺序，或让关键配置使用无歧义 FQDN。删除搜索域前先确认其他业务依赖。

Cheatsheet 短名失败先查 search；`hostname -f` 异常进入 NSS；A 与 AAAA 分开；PTR 是独立区域；短名命中错误要查 search 顺序。

[诊断专题] dig 成功但 NSS 或应用解析失败

“dig 成功”只证明某次 DNS 查询得到答案。要解释应用失败，必须继续比较 NSS、默认 resolver、指定服务器和应用自身状态。最有区分度的第一步，是对同一个 FQDN 同时执行 getent 和 dig。

① [诊断] dig 正确而 getent 返回旧地址

当前证据：

```
dig @192.0.2.53 app.ops.example A → 新地址
getent hosts app.ops.example      → 旧地址
```

优先假设：/etc/hosts 存在过期条目，或 NSS 在 DNS 前使用其他来源。

下一条证据：

```
grep -nF 'app.ops.example' /etc/hosts
grep -E '^[:space:]*hosts:' /etc/nsswitch.conf
```

最小修复：删除或修正唯一的过期条目；保持合理 NSS 策略。不要通过改变 hosts: 行的全局来源顺序来绕开坏数据。

② [诊断] 指定服务器 dig 成功而普通 dig 失败

假设：当前默认 resolver 没有使用该服务器、DNS 顺序不同、本地 stub 异常，或到默认服务器的网络路径失败。

下一条证据：

```
cat /etc/resolv.conf
readlink -f /etc/resolv.conf
nmcli -f ipv4.dns,ipv6.dns connection show exam-net
nmcli -f IP4.DNS,IP6.DNS device show
```

只有在确认持久 profile 错误后才修改 NetworkManager；不要直接覆盖 `resolv.conf` 形成下一次重连必然复发的临时修复。

③ [诊断] 普通 `dig` 成功而 `getent` 完全失败

可能原因：

- `hosts`：行没有可用 DNS 来源；
- NSS 动作在 DNS 前提前 `return`；
- 应用/NSS 需要的地址族没有数据；
- 系统解析调用与 `dig` 使用了不同本地 resolver；
- 名称形式不同，例如一个使用短名，一个使用 FQDN。

下一条证据：固定同一个 FQDN、同一地址族，再比较 `getent ahostsv4/ahostsv6` 和 `dig A/AAAA`。

④ [诊断] `getent` 已正确而长期应用仍使用旧地址

此时不要继续修改 DNS。调查缓存持有者：

应用自身 DNS 缓存
→ 长期运行的语言运行时
→ 本地 `caching resolver`
→ 已建立的长连接或连接池
→ 容器/命名空间中的独立解析配置

最小修复：只 `reload`、`restart` 或刷新已经确认持有旧结果的组件。无调查重启整台主机虽然可能暂时恢复，却破坏证据并扩大影响。

⑤ [诊断] 所有名称工具成功而业务仍失败

名称解析层已经基本通过，下一步转向：

地址可达性
→ 路由
→ 端口监听
→ `firewalld`
→ SELinux
→ TLS/证书
→ 应用协议和认证

这些对象分别归属第 18、20、21、28/29 章或应用自身。不要继续在名称解析层随机修改。

⑥ [安全边界] 缓存没有统一“万能清理命令”

glibc NSS 查询本身不提供一个适用于所有环境的全局缓存清理按钮。缓存可能由 `nscd`、`SSSD`、`systemd-resolved`、`dnsmasq` 或应用维护，而且当前主机未必启用这些组件。必须先识别实际服务，再使用其官方接口。

Cheatsheet `dig` 对、`getent` 错先查 hosts/NSS；指定服务器对、默认路径错先查 resolver；`getent` 对、应用错查应用缓存和连接；名称都对则离开 DNS 层。

[经典任务] 配置静态主机名、本地映射和 DNS 搜索域

环境

系统已经存在并激活 NetworkManager connection profile `exam-net`。IPv4 地址、前缀、网关和路由已经按第 18 章完成，不允许修改。当前主机名来自临时网络配置；DHCP 还提供了一个不符合本题要求的 DNS server。

本任务使用文档保留地址，仅用于练习参数：

```
静态主机名: node1.training.example
DNS server: 192.0.2.53
DNS search: training.example
本地映射: 192.0.2.44 repo.training.example repo-local
```

目标终态

- 当前和持久静态主机名均为 `node1.training.example`；
- `exam-net` 只使用题目指定的 IPv4 DNS server；
- 短名 `node1` 可通过 `training.example` 搜索域解析；
- `repo.training.example` 与别名 `repo-local` 通过 `/etc/hosts` 得到 `192.0.2.44`；
- 不改变现有地址、路由和网关；
- 变更后提供配置、当前状态、查询和功能证据。

限制条件

- 不创建或删除 connection profile；
- 不直接把 `/etc/resolv.conf` 当作持久真源；
- 不使用 `chattr` 锁定 resolver 文件；
- 不部署 DNS 服务器；
- 不随意修改 `hosts` 顺序；
- 重新激活连接前评估远程会话中断风险。

验收矩阵

评分对象	推荐证据	成功边界
静态主机名	<code>hostnamectl status</code> 、 <code>/etc/hostname</code>	当前与持久均正确
profile DNS	<code>nmcli connection show exam-net</code>	<code>server/search/ignore-auto-dns</code> 符合题意

评分对象	推荐证据	成功边界
当前 resolver	device view 与 <code>/etc/resolv.conf</code>	当前路径已更新
hosts 条目	<code>getent hosts repo-local</code>	NSS 返回 <code>192.0.2.44</code>
FQDN	<code>getent hosts node1.training.example</code>	系统解析得到预期地址
短名	<code>getent hosts node1</code>	search 生效
DNS 原始记录	<code>dig @192.0.2.53 node1.training.example A</code>	指定服务器给出预期 A
持久性	profile 与 <code>/etc/hostname</code>	live 重启留待真实环境复验

[经典任务] 诊断 `dig` 正确但应用得到错误地址

已知状态

系统当前出现以下证据：

```
dig @192.0.2.53 app.ops.example A 取得题目给定的新地址
getent hosts app.ops.example      取得旧地址 192.0.2.99
getent hosts app                  失败
```

现有 `exam-net` profile 缺少 `ops.example` search domain；`/etc/hosts` 中残留旧的 `app.ops.example` app 映射。应用长期运行并使用短名 `app`。

任务要求

1. 建立变更前基线；
2. 确认 `hosts:` 实际顺序；
3. 删除或修正唯一的过期本地映射；
4. 在现有 `exam-net` 中配置 `ops.example` search domain；
5. 不通过把 `dns` 移到 `files` 前面掩盖冲突；
6. 分别验证短名、FQDN、DNS 原始记录和应用功能；
7. 若命令行解析已恢复而应用仍使用旧地址，只刷新已经确认持有缓存的应用组件。

典型错误

- 看到 `dig` 成功便结束调查；
- 删除整个 `/etc/hosts`；
- 为一次故障改变全局 NSS 来源顺序；
- 无调查地重启 NetworkManager 或整台主机；
- 把应用缓存问题继续解释为 DNS server 内容错误；

[参考解答] 经典任务一：配置主机名、本地映射和搜索域

参考解答不是唯一命令序列，而是一条可审计的状态迁移链。以下命令未在可控的 RHEL 9 VM 中执行；必须在真实环境按接口名、profile 状态和题目给定记录调整。

① 调查：确认现有对象和变更风险

```
hostnamectl status
hostname
cat /etc/hostname

nmcli connection show --active
nmcli -f connection.id,connection.interface-
name,ipv4.method,ipv4.addresses,ipv4.gateway,ipv4.dns,ipv4.dns-search,ipv4.ignore-auto-
dns \
    connection show exam-net

readlink -f /etc/resolv.conf
cat /etc/resolv.conf
grep -E '^[[:space:]]*hosts:' /etc/nsswitch.conf

grep -nE 'repo-local|repo\.training\.example|192\.0\.2\.44' /etc/hosts
```

调查目的：

- 确认 `exam-net` 确实是活动 profile；
- 保存地址、网关和路由基线，避免越界修改；
- 判断 DNS 是自动获得还是手工指定；
- 确认 resolver 文件由谁管理；
- 避免重复 hosts 条目。

② 操作：设置静态主机名

```
hostnamectl set-hostname node1.training.example
```

立即检查：

```
hostname
hostnamectl status
cat /etc/hostname
```

③ 操作：创建最小 `/etc/hosts` 映射

在保留 localhost 和其他有效条目的前提下，加入：

```
192.0.2.44 repo.training.example repo-local
```

静态检查：

```
grep -nE 'repo-local|repo\.training\.example|192\.0\.2\.44' /etc/hosts
```

NSS 验证：

```
getent hosts repo.training.example
getent hosts repo-local
```

如果 `getent` 仍返回意外地址，继续查 `hosts`：顺序和其他重复条目，不能用 `dig` 代替此步。

④ 操作：修改现有 NetworkManager profile

题目要求只使用指定自动配置，因此：

```
nmcli connection modify exam-net ipv4.dns "192.0.2.53"
nmcli connection modify exam-net ipv4.dns-search "training.example"
nmcli connection modify exam-net ipv4.ignore-auto-dns yes
```

操作后先读取保存配置：

```
nmcli -f ipv4.dns,ipv4.dns-search,ipv4.ignore-auto-dns connection show exam-net
```

⑤ 应用：重新激活正确的 profile

确认远程中断风险后：

```
nmcli connection up exam-net
```

若操作发生在远程主机，应准备控制台或回退路径。本题禁止修改 IP 和路由，因此激活后应再次确认这些前置状态未变化。

⑥ 分层验证

持久配置：

```
cat /etc/hostname
nmcli -f ipv4.dns,ipv4.dns-search,ipv4.ignore-auto-dns connection show exam-net
```

当前有效状态：

```
hostname
nmcli -f GENERAL.CONNECTION,IP4.ADDRESS,IP4.GATEWAY,IP4.DNS,IP4.DOMAIN device show
cat /etc/resolv.conf
```

NSS:

```
getent hosts repo-local
getent hosts node1.training.example
getent hosts node1
```

DNS 层:

```
dig @192.0.2.53 node1.training.example A
```

功能层：使用题目指定的真实应用或服务请求，不用 `ping` 替代全部业务验收。

⑦ 持久性声明

静态证据只能确认命令、配置路径和逻辑闭环；真实环境仍需重启系统，重新检查 `hostname` 与 `exam-net` 自动激活状态，并重复 `getent`、`dig` 和应用功能测试。

[参考解答] 经典任务二：从 `dig`/`getent` 差异定位最小修复

① 复现并固定查询对象

首先对完全相同的 FQDN 和地址类型取证：

```
getent ahostsv4 app.ops.example
dig @192.0.2.53 app.ops.example A
```

不要用 `getent app` 与 `dig app.ops.example` 直接比较，因为名称形式不同会引入 `search` 变量。

② 检查 NSS 策略与本地数据

```
grep -E '^[[:space:]]*hosts:' /etc/nsswitch.conf
grep -nE '^(^|[[:space:]])(app|[[:space:]]|$)|app\.ops\.example' /etc/hosts
```

若 `files` 位于 `dns` 前，而 `/etc/hosts` 中存在旧地址，差异已经得到解释。此时不应调整全局顺序，而应修正坏数据。

③ 最小修改 `/etc/hosts`

仅删除或修正过期的 `app.ops.example app` 条目，保留 `localhost` 和其他有效映射。完成后立即重复：

```
getent ahostsv4 app.ops.example
```

期望结果应与题目指定 DNS 记录一致。若仍不同，继续检查其他 NSS 来源，不要假定只有 `files` 和 `dns`。

④ 添加短名所需 search domain

建立 profile 基线：

```
nmcli -f ipv4.dns,ipv4.dns-search,ipv4.ignore-auto-dns connection show exam-net
```

配置：

```
nmcli connection modify exam-net ipv4.dns-search "ops.example"
```

本题没有说明要抛弃 DHCP DNS，因此不机械设置 `ipv4.ignore-auto-dns yes`。若 profile 还有其他业务所需 search 域，应按题意决定追加还是替换，而不是无条件清空。

应用配置：

```
nmcli connection up exam-net
```

⑤ 重新执行分层验证

```
getent hosts app.ops.example
getent hosts app

dig app.ops.example A
dig @192.0.2.53 app.ops.example A
```

判断边界：

- `getent` FQDN 正确：NSS 冲突已修复；
- `getent` 短名正确：search 已生效；
- 普通 `dig` 与指定服务器结果可解释：默认 resolver 路径正确；
- 应用仍旧：进入应用缓存或连接调查。

⑥ 只刷新确认持有旧结果的应用组件

先查看应用文档、服务状态和日志，确认它是否缓存 DNS 或保持连接池。随后选择应用支持的 reload、连接池刷新或受控 restart。不要：

- 清空未知服务的缓存；

- 重启整台主机代替定位；
- 再次修改已经正确的 DNS 数据；
- 把“进程重启成功”扩大为业务功能已经恢复。

⑦ 最终验收

使用应用本身的请求路径验证目标 FQDN、端口和业务响应，并保留时间、查询名称、返回地址和应用日志证据。若名称解析均正确而业务仍失败，按章节边界转入路由、服务、firewalld、SELinux 或应用层调查。

[本章收束] 从名称字符串回到应用实际使用的地址

本章的核心不是记住一组零散命令，而是形成以下调用路径：

题目中的名称

- 判断短名、FQDN 或绝对名称
- 确认本机 hostname 是否只是身份
- 读取 NSS hosts 策略
- 比较 /etc/hosts 与 DNS
- 检查 NetworkManager 持久 DNS 和当前 resolver
- 分别查询 A、AAAA、PTR
- 用 getent 验证系统解析
- 用应用请求验证最终功能

考试中最常见的得分点是正确设置主机名、hosts 和 DNS；最常见的失分点则是混淆工具边界和持久性。工作中还要进一步记录变更基线、控制远程连接风险、识别缓存持有者，并在名称解析已经正确时及时离开本层。

最终应能快速回答：

- 这是本机身份问题，还是名称到地址的问题？
- 当前值和持久值是否一致？
- 应用走的是 NSS，还是直接 DNS？
- /etc/hosts 和 DNS 谁先返回？
- 短名扩展成了哪个 FQDN？
- A、AAAA、PTR 哪一种记录失败？
- dig 成功为什么仍不能证明应用成功，以及修复后哪条证据真正重复了原始失败路径？

主要判断表

看到的现象	先不要下的结论	下一条最有区分度的证据
hostnamectl 正确	DNS 已正确	getent hosts <FQDN> 与 dig <FQDN> A/AAAA

看到的现象	先不要下的结论	下一条最有区分度的证据
<code>dig</code> 成功	应用一定成功	对同一名称执行 <code>getent</code> ，再重复应用请求
<code>getent</code> 返回旧地址	DNS 一定错误	检查 <code>hosts:</code> 顺序与 <code>/etc/hosts</code> 冲突
FQDN 成功、短名失败	网络不通	读取 <code>search domain</code> ，并查询扩展后的 FQDN
A 成功、PTR 失败	正向记录也错误	<code>dig -x <ADDRESS></code> ，单独确认反向区域
profile 属性正确	当前 resolver 已应用	<code>nmcli device show</code> 、 <code>/etc/resolv.conf</code> 与实际查询
工具均成功、业务失败	继续改 NSS/DNS	应用日志、缓存持有者、连接和服务层证据

工作方法

1. 固定查询对象。记录名称形式、地址族、DNS server 和时间，避免比较不同问题。
2. 先读取再修改。保存 `hostname`、`hosts:`、`/etc/hosts`、`profile` 与 `resolver` 基线。
3. 让差异定位层次。`dig` 与 `getent` 的差异比重复执行同一命令更有价值。
4. 只做最小修复。删除过期条目、补齐 `search` 或修正 `profile`，不改变全局来源顺序来掩盖坏数据。
5. 沿原失败路径再验证。配置正确、当前状态正确、查询正确和应用功能正确必须分别成立。
6. 区分静态证据与运行验证。配置文件和 `connection profile` 只能说明持久输入；真实重连、重启和应用缓存刷新仍需为目标 RHEL 9 环境复验。

向下一章交接

本章把目标名称解析成了地址，但 SSH 连接还要回答另一组问题：客户端究竟连接哪台主机、服务端怎样监听、用户怎样认证、主机密钥怎样证明远端身份。第 20 章《SSH 客户端、服务端与密钥认证》将从这里继续；不要把 SSH 主机密钥警告误判为 DNS 解析失败，也不要改 `/etc/hosts` 绕过尚未调查的主机身份变化。